# **Session 3 Date:** April 29, 2026 (2 hours)

**Objective:**
Implement the SSSP and validate it by using the bellman-ford style bsp relaxation.

**Attempts made:**

SSSP incorporated the BFS structure, while slack Worked on adding weighted distances to the SSSP. The main difficulty was selecting a sentinel value for vertices determined to be permanently reached.

Previously, I used INT_MAX, which resulted in an overflow edge case when a weight was added. Also, for the first two iterations, I also neglected the boundary check, and added edge weight to the vertex, resulting in some unexplained vertex states.

**Solutions/Workarounds:**

I decided that weights would add only if “prev[u]” was not “LLONG_MAX”, and used “LLONG_MAX” as the sentinel:

```
if (prev[u] != LLONG_MAX) {
   long long candidate = prev[u] + w;

   if (best == LLONG_MAX || candidate < best)

      best = candidate;

}
```

I then went ahead and made a fun little helper function to distance() that would replace LLONG_MAX with -1 when outputting:

```
long long distance(uint32_t v) const {
   return (dist[v] == LLONG_MAX) ? -1 : dist[v];
}
```

**Commits:**

added SSSP algorithm, wired SSSP into main with serial vs parallel validation

**Tests/Analysis:**

I made a weighted test case utilizing the graph below:

0 1 5
0 2 1
2 1 1
1 3 1
2 3 10

I received the below output:

SSSP serial vs parallel match ✔

SSSP distances:

vertex 0 = 0

vertex 1 = 2

vertex 2 = 1

vertex 3 = 3

The output vertex 1 was equal to 2 because the path with the weights of 0 to 2 to 1 was equal to 1 and 1, totaling 2. The path directly from 0 to 1 was equal to 5. As such, the output vertex 1 confirmed that the algorithm was actually computing the cheapest path.

The output of vertex 3 was equal to 3, and was because of the path from 0 to 2 to 1 to 3 totaling 11, compared to the path from 1 to 3 to 2 to 3, which was 10. The outputs for the serial and parallel algorithms were exact.

**Learning outcome:**

You can be certain first member values need to be handled more cautiously as the enhancements of INT_MAX values include the effects of Weight wrapping to the NODE values causing the values to be State Variables.